

1. ¿Qué es una Build Tool en Java?

Una herramienta de construcción “Build Tool” es un programa que automatiza las tareas necesarias para transformar el código fuente en algo ejecutable o distribuible (como un archivo jar o war). Se encarga de compilar el código, gestionar y descargar dependencias externas, ejecutar pruebas automatizadas, empaquetar la aplicación y desplegarla en entornos de destino sin intervención manual en cada paso.

2. Herramientas principales:

Apache Ant es una herramienta Java que utiliza archivos XML basados en tareas. Es altamente procedural y flexible, pero carece de convenciones estrictas y no gestiona dependencias de forma nativa (requiere herramientas adicionales como Apache Ivy).

Apache Maven: Introdujo el principio de convención sobre configuración y la gestión declarativa de dependencias mediante repositorios centrales y locales. Organiza todo el ciclo de vida del software a través de un archivo central XML llamado pom.xml.

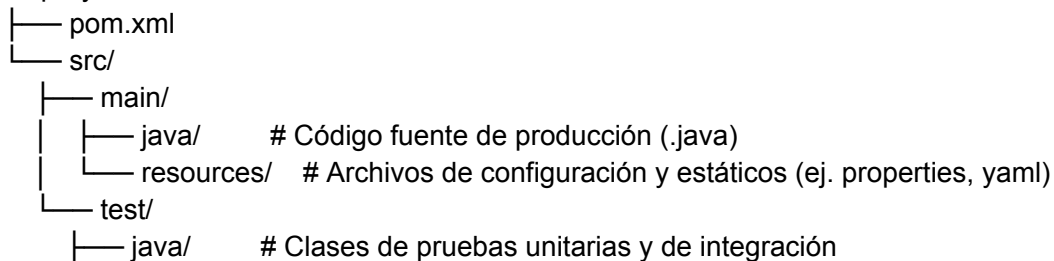
Gradle: Combina la flexibilidad de Ant con las convenciones y gestión de dependencias de Maven. Utiliza un Domain-Specific Language (DSL) basado en Groovy o Kotlin, e implementa compilación incremental y un demonio en segundo plano para optimizar el rendimiento.

3. Apache Maven en detalle

Estructura de directorios estándar

Maven sigue una convención estricta para organizar los archivos:

mi-proyecto/



¿Qué es el archivo POM de Maven?

El **POM** “project object model” en el archivo pom.xml, es la unidad central de configuración de un proyecto Maven. Contiene:

Las coordenadas del proyecto (groupId, artifactId, version).

La lista de dependencias y librerías externas que Maven descarga automáticamente.

La configuración de plugins para extender tareas de compilación, empaquetado o reporte.

Parámetros globales y propiedades (como la versión de Java del compilador).

Propósito del archivo application.properties

No es un archivo propio del núcleo de Maven, sino el archivo de configuración estándar en frameworks del ecosistema Java (como Spring Boot o Quarkus) ubicado en `src/main/resources/`. Su propósito es externalizar la configuración de la aplicación en tiempo de ejecución, centralizando parámetros como:

Conexiones a bases de datos (URL, usuario, contraseña).

Puerto y contexto del servidor web embebido.

Perfiles de entorno (desarrollo, testing, producción) y niveles de logging.

Build Lifecycle de Maven y sus etapas

Maven organiza el proceso en tres ciclos de vida predefinidos: **default** (construcción y despliegue), **clean** (limpieza del directorio target/) y **site** (generación de documentación).

El ciclo **default** ejecuta secuencialmente las siguientes fases clave:

Validate: Valida que el proyecto esté completo y que toda la información requerida en el POM esté disponible.

Compile: Compila el código fuente (`src/main/java`) en archivos binarios `.class`.

Test: Ejecuta las pruebas unitarias usando frameworks como JUnit o TestNG sobre el código compilado (estas pruebas no requieren empaquetar la aplicación).

Package: Toma el código compilado y lo empaqueta en su formato final distribuible (`.jar`, `.war`).

Verify: Ejecuta pruebas de integración y comprueba que se cumplan los criterios de calidad.

Install: Copia el paquete generado en el repositorio local (`~/.m2/repository`) para que otros proyectos locales puedan usarlo como dependencia.

Deploy: Copia el artefacto final a un repositorio remoto compartido para su distribución en equipos o entornos de producción.